Internal & Stakeholder Brief | v1.0

Claude Code Productivity Stack

A cost-engineered AI development environment: architecture, token economics & operations

Executive summary

This is an opinionated configuration of Claude Code (Anthropic's agentic coding CLI) tuned for a single objective: **maximum capability per token spent.** It layers a small, always-on set of "skills" with heavyweight automation kept dormant until explicitly summoned, a three-tier persistent memory so the assistant never re-learns context, on-demand code-graph navigation, and analysis offloaded to free external compute. The result is an assistant that compounds knowledge across sessions while holding recurring cost near a fixed floor; the entire setup is version-controlled and restorable on a new machine in minutes.

~2%
of the context window consumed by the always-on layer per session (≈3–4k of 200k tokens)

0
idle cost from heavyweight harnesses, installed but disabled until needed

~71% / ~53%
fewer lines of code / lower cost on a top-tier model with the efficiency skill (benchmark)

7–70×
fewer tokens to navigate a large codebase via a local knowledge-graph vs. blind search

Architecture at a glance

ALWAYS-ON (CHEAP)

Lazy-loaded skills: only a one-line description sits in context until a skill fires. Methodology (TDD, debugging, planning, parallel agents), an efficiency ruleset, UI/UX intelligence, document generation, knowledge-base & research tooling.

ON-DEMAND (DORMANT → 0 IDLE)

Multi-agent "swarm" orchestration, spec-driven workflow engine, and vertical skill packs (marketing, SEO, finance, legal). Installed but **disabled**; enabled per task, disabled after. Avoids per-tool-call overhead.

THREE-LAYER MEMORY

Native (auto-recalled facts, ~0 extra tokens) → **local knowledge vault** (human-browsable notes) → **external notebook** (deep archive, queried on demand). Tiered recall always tries the cheapest source first.

NAVIGATION & OFFLOAD

A local **code knowledge-graph** lets the assistant traverse a repo by structure instead of reading everything. Heavy analysis & media generation are **offloaded to free external compute**, consuming no model tokens.

OPERATING LOOP

Kickoff: pick tools for the project → Work: code / research / multi-agent build → Tiered recall: cheapest memory first → Wrap-up: persist to all 3 memory layers → Backup: version everything

Token economics & measured impact

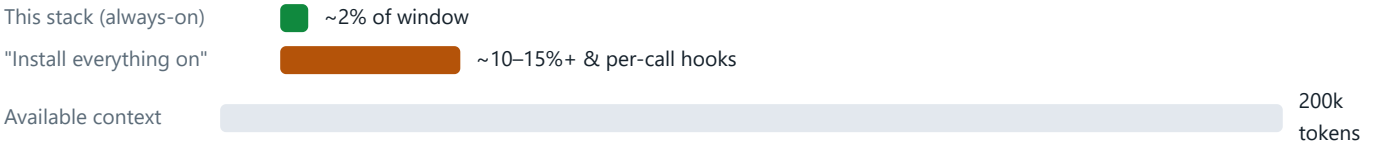
LEVER	EFFECT	SOURCE
Always-on layer footprint	≈ 3–4k tokens/session (~2% of a 200k window); skills lazy-load their bodies only when invoked	Measured

LEVER	EFFECT	SOURCE
Heavyweight harnesses kept disabled	0 idle cost vs. an estimated 10–30k tokens/session + per-tool-call hook overhead if left always-on	Measured / est.
Efficiency skill (lazy-coding ruleset)	~71% fewer lines of code, ~53% lower cost, ~71% faster on a top-tier model; one sample task fell from ≈$139 to ≈$38	Published & reproduced
Code knowledge-graph navigation	7–70× fewer tokens than reading/searching a large repo; a real 28-file module built to 561 nodes / 943 edges fully offline (no API)	Measured
Persistent memory (recall vs. re-explain)	Replaces an estimated 5–50k tokens of context re-explanation per session with ~0 (native) or a single offloaded query	Estimate
External analysis/media offload	Heavy synthesis & deliverables (audio, infographics, slides) run on free external compute, 0 model tokens	By design

EFFICIENCY SKILL: REDUCTION ON A TOP-TIER MODEL

Third-party benchmark, independently reproduced on a top-tier model. Gains are smaller on lightweight models; this stack targets the high-capability tier where verbosity (and therefore the savings) is largest.

RECURRING COST: ALWAYS-ON FLOOR VS. CONTEXT BUDGET



How it's used (operations)

DAILY LOOP

Open the project → a start-of-session prompt recommends a token-efficient toolset → work with methodology skills and, for big builds, a multi-agent team → wrap up to persist decisions → one-command backup.

HEAVY CODING

A router keeps parallel work on cheap built-in sub-agents by default and only escalates to the high-cost "swarm" for genuinely large builds, then powers it back down.

RESEARCH PIPELINE

One command: gather sources → offload analysis & deliverables to free external compute → file linked notes into the knowledge vault. Knowledge compounds instead of scattering.

MEMORY UPKEEP

A periodic consolidation pass merges duplicates, fixes stale facts, and prunes the index, keeping per-session memory cost flat even as the knowledge base grows for years.

Business value

Lower run-rate	Token spend held near a fixed floor; the biggest cost drivers are off by default and the coding model is constrained to write less.
Faster delivery	Up to ~70% faster on constrained coding tasks; no time lost re-explaining context to a stateless assistant.
Compounding knowledge	Every session enriches a persistent, searchable knowledge base: institutional memory that survives staff and session turnover.
Portable & resilient	The entire environment is version-controlled and restored on a new machine in minutes via one script, with no lock-in and no manual rebuild.

Governed & safe	Third-party tools are source-verified before install; secrets and private data are excluded from all backups by policy.

Portability & governance

The stack is captured as a version-controlled bundle: a single idempotent bootstrap script reinstalls every plugin, skill, tool, and connector and restores configuration with machine-local paths, with no personal data embedded. Credentials, authentication cookies, and private notes are deliberately kept out of the backup. New third-party components are verified for authenticity (author, popularity, typosquat checks) before they are allowed to run, and the most expensive automation requires explicit opt-in each time it is used.

Primary lever Cost saving Prepared as an internal & stakeholder overview. Figures labeled "Measured" were observed in this environment; "Published & reproduced" come from a third-party benchmark re-run locally; "Estimate" are conservative working figures. No personal, client, or credential data is included.